

FASTAPI + SQLALCHEMY

Ultimate 1-Page Quick Reference & Architecture Cheat Sheet

DESKTOP REFERENCE

01. FASTAPI CORE

Q: What is FastAPI?

Modern Python web framework based on **Starlette** (ASGI/web) & **Pydantic** (validation/data).

Q: Why is it fast?

Async via **ASGI** + **async/await**, lightweight **Starlette** core, and fast **Pydantic** schema parsing.

Q: What is **Uvicorn** & **ASGI**?

ASGI is the standard interface for async Python servers. **Uvicorn** is the high-performance ASGI server executing FastAPI.

02. REST ENDPOINTS & CODES

```
GET    /products    → List all
GET    /products/1  → Get single
POST   /products    → Create
PUT    /products/1 → Replace/Update
DELETE /products/1  → Delete
```

HTTP Status Codes:

200 OK Success	401 Unauthenticated
201 Created	403 Forbidden
204 No Content	404 Not Found
400 Bad Request	422 Validation Error
500 Internal Server Error	

03. PYDANTIC VALIDATION

Used for **validating**, **parsing** & **serializing** API data.

```
class ProductCreate(BaseModel):
    name: str
    price: float
```

Q: Why separate **Schemas** from **DB Models**?

Separates **API contract** from DB structure, enforces input validation, and prevents exposing internal secrets.

Request JSON → Pydantic Schema → Validated Data

07. CONFIGURATION & SECRETS

.env → Pydantic Settings → Application

Why .env? Keeps secrets (DATABASE_URL, JWT_SECRET) outside source control. Never commit secrets!

04. SQLALCHEMY ORM

Python SQL toolkit & **Object-Relational Mapper** mapping Python classes to database tables.

Python Class → Table | Python Object → DB Row

Core Concepts:

Base	Base for ORM models
Mapped[T]	Typed ORM field
mapped_column()	DB column specs
AsyncSession	Async DB session
select().where()	Build query & filter
scalars().all()	Extract ORM objects
commit()/refresh()	Save txn / reload DB

EF Core vs SQLAlchemy:

EF CORE (.NET)	SQLALCHEMY (PY)
DbContext	→ AsyncSession
Entity	→ ORM Model
LINQ	→ SQLAlchemy expr
Add() / Remove()	→ add() / delete()
SaveChangesAsync()	→ commit()
ToListAsync()	→ scalars().all()
FirstOrDefaultAsync()	→ scalar_one_or_none()

05. ASYNC / AWAIT I/O

DB operations are **I/O-bound**. Async lets the server handle other requests while waiting for DB responses.

```
res = await session.execute(stmt)
await session.commit()
```

Req → DB I/O (Waiting) → Server handles other reqs!

06. ALEMBIC MIGRATIONS

Database schema migration tool for versioning.

SQLAlchemy Model → Migration → DB

Why not create `all()`? It only creates missing tables, not tracking controlled schema version changes over time.

EF Core vs Alembic Commands:

Add-Migration	→ revision --autogenerate
Update-Database	→ upgrade head
__EFMigrationsHistory	→ alembic_version

08. APP ARCHITECTURE

Production Clean Layering:

Request → Router/API → Schema Validation
↓
Service Layer (Business Logic)
↓
Repository Layer → SQLAlchemy → PostgreSQL

09. TOP 10 INTERVIEW QS

- FastAPI?** ASGI framework powered by **Starlette** & **Pydantic**.
- Why fast?** Async I/O + **Starlette** speed + **Pydantic** validation.
- Pydantic?** Data validation, parsing, and serialization.
- SQLAlchemy?** Python SQL toolkit & ORM engine.
- ORM?** Maps Python objects to DB tables & rows.
- AsyncSession?** Non-blocking SQLAlchemy DB session handler.
- Alembic?** Version-controlled DB schema migration tool.
- Schema vs Model?** API Contract (Pydantic) vs DB Schema (ORM).
- Why .env?** Isolates secrets/configs outside Git codebase.
- Prod Stack?** Router → Service → Repository → ORM → DB.